

Example: WordCount v2.0

WordCount version one works well with files that only contain words. However, see what happens if you remove the current input files and replace them with something slightly more complex.

```
$ hadoop fs -rm /user/cloudera/wordcount/input/*
```

Following are three text files that you can add to your input directory.

HadoopFile0.txt:

```
Hadoop is the Elephant King!
A yellow and elegant thing.
He never forgets
Useful data, or lets
An extraneous element cling!
```

HadoopFile1.txt:

```
A wonderful king is Hadoop.
The elephant plays well with Sqoop.
But what helps him to thrive
Are Impala, and Hive,
And HDFS in the group.
```

HadoopFile2.txt:

```
Hadoop is an elegant fellow.
An elephant gentle and mellow.
He never gets mad,
Or does anything bad,
Because, at his core, he is yellow.
```

You can create the files however you like. You can use the following shell commands, or you can use the Makefile command "make poetry").

```
$ echo -e "Hadoop is the Elephant King! \nA yellow and elegant thing.\nHe never forgets\nUseful data, or lets\nAn extraneous element cling! " > HadoopPoem0.txt
$ echo -e "A wonderful king is Hadoop.\nThe elephant plays well with Sqoop.\nBut what helps him to thrive\nAre Impala, and Hive,\nAnd HDFS in the group." > HadoopPoem1.txt
$ echo -e "Hadoop is an elegant fellow.\nAn elephant gentle and mellow.\nHe never gets mad,\nOr does anything bad,\nBecause, at his core, he is yellow." > HadoopPoem2.txt
$ hadoop fs -put HadoopP* /user/cloudera/wordcount/input
$ rm HadoopPoem*
```

Remove the previous results and the application with the new text files as input.

```
$ hadoop fs -rm -r -f /user/cloudera/wordcount/output
$ hadoop jar wordcount.jar org.myorg.WordCount /user/cloudera/wordcount/input /user/cloudera/wordcount/output
```

Now, view the results.

```
$ hadoop fs -cat /user/cloudera/wordcount/output/*
```

You notice that there are some problems with the output. The WordCount application counts lowercase words separately from words that start with uppercase letters, even though they are the same word.

```
. . .
Elephant    1
elephant    2
. . .
```

Hadoop also considers punctuation and small words significant.

```
. . .
!    2
,    7
.    7
A    2
An   2
And  1
. . .
```

You can update your sample code to address these problems and return a more accurate count.

Removing Case Sensitivity

You can add a few lines of code to remove case sensitivity, so that both lowercase and capitalized versions of your words are included in a single count.

You can find source code for the three versions of WordCount at <http://tiny.cloudera.com/hadoopTutorialSample>.

The following describes the changes in WordCount version 2.

Line 5: Import the Configuration class. You use it to access command-line arguments at run time.

```
import org.apache.hadoop.conf.Configuration;
```

Line 47: Create a variable for the case sensitivity setting in the Map class.

```
private boolean caseSensitive = false;
```

Lines 50 - 55: Add a setup method. Hadoop calls this method automatically when you submit a job. This code instantiates a Configuration object, and then sets the class caseSensitive variable to the value of the wordcount.case.sensitive system variable set from the command line. If you don't set a value, the default is false.

```
protected void setup(Mapper.Context context)
    throws IOException,
    InterruptedException
{
    Configuration config = context.getConfiguration();
    this.caseSensitive = config.getBoolean("wordcount.case.sensitive", false);
}
```

Lines 60 - 62: You turn off case sensitivity here. If caseSensitive is false, the entire line converts to lowercase before it is parsed by the StringTokenizer.

```
if (!caseSensitive) {
    line = line.toLowerCase();
}
```

Running WordCount Version Two

Follow these steps to run the updated version.

1. Rebuild the application. You can enter these instructions at the command line, or you can use the Makefile command `make jar` if you are using a CDK package installation.

To compile in a package installation:

```
$ rm -rf build wordcount.jar
$ mkdir -p build
$ javac -cp /usr/lib/hadoop/*:/usr/lib/hadoop-mapreduce/* WordCount.java -d build -Xlint
$ jar -cvf wordcount.jar -C build/ .
```

To compile with a parcel installation:

```
$ rm -rf build word_count.jar
$ mkdir -p build
$ javac -cp /opt/cloudera/parcels/CDH/lib/hadoop/*:/opt/cloudera/parcels/CDH/lib/hadoop-mapreduce/* \
    WordCount.java -d build -Xlint
$ jar -cvf word_count.jar -C build/ .
```

2. Remove the previous results.

```
$ hadoop fs -rm -r -f /user/cloudera/wordcount/output
```

3. Run the application. By default, words are converted to lowercase before being counted.

```
$ hadoop jar wordcount.jar org.myorg.WordCount /user/cloudera/wordcount/input /user/cloudera/wordcount/output
```

When you look at the results, you see that all words have been converted to lowercase. Now you get three lowercase elephants.

```
$ hadoop fs -cat /user/cloudera/wordcount/output/*
. . .
elephant    3
. . .
```

4. To turn case sensitivity on, pass the system variable `-Dwordcount.case.sensitive=true` on the command line at run time.

```
$ hadoop fs -rm -f -r /user/cloudera/wordcount/output
$ hadoop jar wordcount.jar org.myorg.WordCount -Dwordcount.case.sensitive=true /user/cloudera/wordcount/input
/user/cloudera/wordcount/output
```

View the results, and see one uppercase *Elephant* and two lowercase *elephants*.

```
. . .
```

```
$ hadoop fs -cat /user/cloudera/wordcount/output/*  
Elephant      1  
elephant      2  
* * *
```

Page generated September 2, 2015.

[<< Other Features](#)

[Example: WordCount v3.0 >>](#)

©2015 Cloudera, Inc. All rights reserved

[Terms and Conditions](#) [Privacy Policy](#)